



ESCUELA TÉCNICA SUPERIOR
DE INGENIERÍA INFORMÁTICA

GRADO EN INTELIGENCIA ARTIFICIAL

Práctica obligatoria 1: Detección de objetos

Visión Artificial

Iván de Rada López
Raúl Vicente Sánchez
Hugo Salvador Aizpún

Curso 2025–2026

Índice

1	Contexto de la práctica	2
2	Desarrollo de la práctica	3
2.1	Herramientas y organización del proyecto	3
2.2	Algoritmo principal: detector MSER y validación HSV	3
2.3	Detectores complementarios para casos específicos	5
2.4	Validación final por densidad de bordes	7
2.5	Eliminación de detecciones repetidas	8
2.6	Formato de salida y scripts de apoyo	10
3	Evaluación de los resultados obtenidos	11
3.1	Resumen cuantitativo	11
3.2	Curvas precision-recall	12
3.3	Imágenes de evaluación	13
3.4	Discusión de resultados	14
4	Conclusión	14
5	Bibliografía	15

1. Contexto de la práctica

La práctica consiste en desarrollar un sistema de detección de paneles informativos de autopista en imágenes reales tomadas desde un vehículo. Estos paneles presentan características visuales que permiten diferenciarlos del resto de elementos de la escena: tienen forma generalmente rectangular, un color azul dominante, borde claro y una distribución espacial compacta.

El objetivo no es reconocer el contenido interno del panel, sino localizar las regiones rectangulares azules correspondientes a los subpaneles informativos. Para ello, el sistema debe generar cajas delimitadoras o *bounding boxes* sobre las imágenes de test y asignar a cada detección una puntuación de confianza o *score*.



Figura 1: Ejemplo de paneles informativos de autopista utilizados como referencia visual.

El enunciado propone distintas estrategias de detección basadas en propiedades visuales del objeto:

- detección de regiones de alto contraste
- detección de formas rectangulares
- localización de píxeles azules y análisis de su distribución espacial
- detección de formas internas características, como flechas o símbolos
- combinación de información de color y forma

La práctica establece una restricción metodológica importante: solo se pueden emplear técnicas clásicas de procesamiento de imagen, sin utilizar clasificadores ni librerías de aprendizaje automático. Por este motivo, la solución desarrollada combina información de contraste, color, forma geométrica, textura interna y posición espacial.

El flujo general del sistema se organiza en varias etapas. En primer lugar, se obtienen regiones candidatas mediante MSER. A continuación, cada región candidata se valida mediante una máscara de color en el espacio HSV. Después se combinan esas detecciones con dos estrategias complementarias basadas en bordes, contornos y una máscara HSV

más permisiva para imágenes con niebla. Finalmente, las detecciones se validan mediante densidad de bordes y se filtran con un algoritmo de supresión de repetidas basado en IoU y contención.

2. Desarrollo de la práctica

2.1. Herramientas y organización del proyecto

La implementación se ha realizado en Python utilizando principalmente OpenCV y NumPy. En la versión final se ha organizado el código en dos archivos principales, además del script de evaluación proporcionado por la asignatura.

- **main.py**: procesa las imágenes del directorio de test, llama al detector, genera el fichero `resultado.txt` y guarda las imágenes con las detecciones dibujadas.
- **detector_paneles.py**: contiene la clase `DetectorPaneles` y las funciones auxiliares de IoU, contención y eliminación de detecciones repetidas. En este archivo se concentra la lógica del detector.
- **evaluar_resultados.py**: script proporcionado con la práctica para calcular las curvas *precision-recall* y la métrica AP.

Las principales herramientas empleadas han sido:

- **OpenCV**: lectura y escritura de imágenes, conversión entre espacios de color, normalización de contraste, detector MSER, Canny, extracción de contornos, operaciones morfológicas y dibujo de resultados.
- **NumPy**: creación de máscaras binarias, definición de umbrales y operaciones numéricas.

2.2. Algoritmo principal: detector MSER y validación HSV

El detector principal sigue la estructura propuesta en el enunciado de la práctica. La imagen original se convierte primero a escala de grises mediante `cv2.cvtColor`, ya que MSER trabaja sobre regiones de intensidad. Después se aplica una normalización lineal de contraste con `cv2.normalize`, llevando los valores de intensidad al rango 0–255. Esta normalización ayuda a reducir diferencias de iluminación entre imágenes y facilita la detección de regiones estables.

Antes de aplicar MSER se realiza también un suavizado gaussiano de 5×5 , que reduce ruido y pequeños detalles de la imagen.

```
1 gray = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
2 gray = cv2.normalize(gray, None, 0, 255, cv2.NORM_MINMAX)
3 gray_blur = cv2.GaussianBlur(gray, (5, 5), 0)
```

Listing 1: Preprocesado de la imagen antes de aplicar MSER.

A continuación, se aplica MSER con los parámetros `delta=5`, `min_area=600` y `max_area=90000`. MSER permite detectar regiones de alto contraste relativamente estables ante variaciones de intensidad. En este problema resulta adecuado porque los paneles presentan una zona azul homogénea y diferenciada del borde y del fondo.

```
1 self.mser = cv2.MSER_create(delta=5, min_area=600, max_area=90000)
2 regiones, _ = self.mser.detectRegions(gray_blur)
```

Listing 2: Inicialización y aplicación del detector MSER.

Cada región devuelta por MSER se transforma en una caja rectangular mediante `cv2.boundingRect`. Después se aplican varios filtros geométricos para descartar candidatos poco probables. Se eliminan regiones situadas demasiado arriba o demasiado abajo en la imagen, ya que los paneles suelen aparecer en la zona superior-media de la escena, pero no pegados al borde superior. También se descartan regiones poco compactas mediante la proporción entre puntos de la región MSER y el área de su caja envolvente.

```
1 for region in regiones:
2     x, y, w, h = cv2.boundingRect(region)
3
4     if y < alto_img * 0.05 or y > alto_img * 0.65:
5         continue
6
7     if (len(region) / float(w * h)) < 0.45:
8         continue
9
10    relacion_aspecto = w / float(h)
11    if 0.8 < relacion_aspecto < 3.2:
12        mw, mh = int(w * 0.08), int(h * 0.08)
13        x1, y1 = max(0, x - mw), max(0, y - mh)
14        x2 = min(ancho_img, x + w + mw)
15        y2 = min(alto_img, y + h + mh)
```

Listing 3: Obtención de cajas y filtrado geométrico inicial.

La ampliación del 8% en anchura y altura permite incluir el borde blanco del panel. Esto es importante porque MSER tiende a detectar principalmente la región azul interior, por lo que la caja inicial puede quedarse demasiado ajustada y no cubrir completamente el panel anotado.

Posteriormente, cada región candidata se recorta de la imagen original y se redimensiona a un tamaño fijo de 80×40 píxeles. Esta normalización permite evaluar todos los candidatos

en una misma escala. Sobre la región redimensionada se realiza una conversión al espacio HSV y se genera una máscara binaria con el rango de azul saturado definido en el detector.

```
1 self.tamano_base = (80, 40)
2 self.lower_blue = np.array([100, 130, 40])
3 self.upper_blue = np.array([140, 255, 255])
4
5 roi = imagen[y1:y2, x1:x2]
6 roi_resized = cv2.resize(roi, self.tamano_base)
7 hsv = cv2.cvtColor(roi_resized, cv2.COLOR_BGR2HSV)
8 mask = cv2.inRange(hsv, self.lower_blue, self.upper_blue)
9
10 score = cv2.countNonZero(mask) / float(self.tamano_base[0] *
    self.tamano_base[1])
```

Listing 4: Validación de candidatos mediante máscara HSV.

Los píxeles que se encuentran dentro de ese rango se consideran píxeles azules saturados. La puntuación del candidato se calcula como la proporción de píxeles azules respecto al número total de píxeles de la ventana normalizada. Esta medida equivale a una correlación con una máscara ideal en la que se espera que una gran parte de la región pertenezca al color azul del panel.

Si la proporción de píxeles azules supera el umbral 0.40, la región se acepta como detección preliminar.

```
1 if score > 0.40:
2     detecciones_iniciales.append({
3         'box': [x1, y1, x2, y2],
4         'score': score
5     })
```

Listing 5: Aceptación de una detección preliminar según su score.

2.3. Detectores complementarios para casos específicos

Además del detector principal basado en MSER, se han añadido dos estrategias complementarias para mejorar la detección en situaciones donde el método principal puede fallar.

Detección de paneles grandes, borrosos o segmentados

La primera estrategia complementaria está basada en bordes y contornos. Este método está orientado a localizar paneles grandes o regiones en las que MSER no devuelve una caja suficientemente representativa. Para ello, la imagen se convierte a escala de grises, se sua-

viza con un filtro gaussiano y se aplica el detector de bordes Canny con umbrales 30 y 100. A partir del mapa de bordes se extraen contornos externos mediante `cv2.findContours`.

```
1 gray = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
2 blur = cv2.GaussianBlur(gray, (5, 5), 0)
3
4 bordes = cv2.Canny(blur, 30, 100)
5 contornos, _ = cv2.findContours(bordes, cv2.RETR_EXTERNAL,
6                                cv2.CHAIN_APPROX_SIMPLE)
```

Listing 6: Extracción de bordes y contornos para candidatos grandes.

Cada contorno se transforma en una caja rectangular y se filtra por área, compacidad, posición en la imagen y relación de aspecto. Después, la región candidata se valida mediante una máscara HSV de azul, utilizando un rango más permisivo que el detector principal. Este método permite recuperar algunos paneles grandes o parcialmente degradados que no son detectados correctamente por MSER.

```
1 area = cv2.contourArea(cnt)
2 if area > 4000:
3     x, y, w, h = cv2.boundingRect(cnt)
4     extent = area / float(w * h)
5
6     if extent > 0.60 and x > 5 and y > 5:
7         relacion_aspecto = w / float(h)
8
9         if 0.8 < relacion_aspecto < 3.2 and y < alto_img * 0.6:
10             roi = imagen[y:y+h, x:x+w]
11             roi_resized = cv2.resize(roi, self.tamano_base)
12             hsv = cv2.cvtColor(roi_resized, cv2.COLOR_BGR2HSV)
13
14             lower_blue_relajado = np.array([90, 40, 40])
15             mask = cv2.inRange(hsv, lower_blue_relajado, self.upper_blue)
```

Listing 7: Filtrado de contornos y validación por color.

Detección en imágenes con niebla o baja saturación

La segunda estrategia complementaria está orientada a escenas con niebla o baja saturación. En estas imágenes, el color azul de los paneles puede aparecer menos saturado o más cercano al gris, por lo que el umbral HSV estricto del detector principal puede rechazar detecciones correctas.

Para estos casos se analiza únicamente la mitad superior de la imagen, donde normalmente se encuentran los paneles de tráfico, y se utiliza un rango HSV más permisivo. En la versión final se aumentó ligeramente el umbral inferior de saturación respecto a pruebas anteriores para reducir falsos positivos en zonas de cielo o niebla.

```
1 roi_superior = imagen[0:int(alto_img * 0.5), 0:ancho_img]
2
3 lower_blue_fog = np.array([95, 30, 50])
4 upper_blue_fog = np.array([135, 110, 220])
5
6 hsv = cv2.cvtColor(roi_superior, cv2.COLOR_BGR2HSV)
7 mask = cv2.inRange(hsv, lower_blue_fog, upper_blue_fog)
```

Listing 8: Rango HSV permisivo para escenas con niebla.

Sobre la máscara resultante se aplican operaciones morfológicas de apertura y cierre con un kernel de 9×9 . La apertura ayuda a eliminar ruido aislado, mientras que el cierre permite consolidar regiones parcialmente fragmentadas.

```
1 kernel = np.ones((9, 9), np.uint8)
2 mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
3 mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

Listing 9: Limpieza morfológica de la máscara en escenas con niebla.

Finalmente, se extraen contornos y se calcula la densidad de la máscara dentro de cada caja candidata. Si la región tiene suficiente densidad de píxeles compatibles con el color azul degradado, se acepta como detección. En caso contrario, se descarta para evitar falsos positivos producidos por nubes, reflejos u otras regiones de color similar.

```
1 roi_mask = mask[y:y + h, x:x + w]
2 densidad = cv2.countNonZero(roi_mask) / float(w * h)
3
4 if densidad > 0.50:
5     detecciones_niebla.append({
6         'box': [x, y, x + w, y + h],
7         'score': densidad
8     })
```

Listing 10: Cálculo de densidad de máscara para validar candidatos.

2.4. Validación final por densidad de bordes

Una modificación añadida en la versión final consiste en validar las detecciones combinadas mediante densidad de bordes. La idea es que un panel real no suele ser una región completamente lisa: contiene texto, flechas, iconos, bordes internos y cambios de intensidad. En cambio, algunas regiones del cielo, montañas o zonas de niebla pueden tener color parecido al azul del panel, pero apenas presentan textura interna.

Para aplicar este filtro, se recorta cada caja candidata, se convierte a escala de grises y se calcula un mapa de bordes con Canny. Después se mide la proporción de píxeles de borde

dentro de la región. Si esta proporción es superior a 0.012, la detección se mantiene; en caso contrario, se descarta.

```
1 detecciones_validadas = []
2 for det in detecciones_totales:
3     x1, y1, x2, y2 = det['box']
4     roi = imagen[y1:y2, x1:x2]
5     if roi.size == 0:
6         continue
7
8     roi_gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
9     bordes = cv2.Canny(roi_gray, 30, 100)
10    densidad_bordes = cv2.countNonZero(bordes) / float(roi.shape[0] *
11    roi.shape[1])
12
13    if densidad_bordes > 0.012:
14        detecciones_validadas.append(det)
```

Listing 11: Validación de detecciones mediante densidad de bordes.

Este filtro ayuda a reducir falsos positivos sin introducir aprendizaje automático, ya que se basa únicamente en características de imagen calculadas con operadores clásicos.

2.5. Eliminación de detecciones repetidas

Debido a que el sistema combina varias estrategias de detección, es habitual obtener más de una caja para un mismo panel. Además, MSER puede detectar varias regiones dentro de un mismo objeto, especialmente la zona azul interior, el borde o elementos internos del panel. Para evitar que estas detecciones repetidas aparezcan en el resultado final, se ha implementado un algoritmo de supresión de no máximos o NMS.

El procedimiento comienza asignando a cada detección una puntuación ponderada para NMS. Esta puntuación parte del *score* original, pero añade un pequeño factor relacionado con el área de la caja. Con ello se intenta evitar que una detección interna pequeña, como una flecha o un símbolo, tenga prioridad sobre una caja mayor que representa mejor el panel completo.

```
1 for det in detecciones:
2     ancho = det['box'][2] - det['box'][0]
3     alto = det['box'][3] - det['box'][1]
4     area = ancho * alto
5     det['score_nms'] = det['score'] * (1.0 + (area / 100000.0))
6
7 detecciones = sorted(detecciones,
8                     key=lambda x: x['score_nms'],
9                     reverse=True)
```

Listing 12: Puntuación ponderada utilizada en la fase NMS.

A continuación, se selecciona la detección con mayor puntuación ponderada y se compara con el resto de cajas. Una caja se descarta si presenta un solapamiento elevado con una caja ya seleccionada o si aparece mayoritariamente contenida dentro de otra.

```
1 def calcular_iou_y_contencion(box1, box2):
2     x_izq = max(box1[0], box2[0])
3     y_arriba = max(box1[1], box2[1])
4     x_der = min(box1[2], box2[2])
5     y_abajo = min(box1[3], box2[3])
6
7     if x_der <= x_izq or y_abajo <= y_arriba:
8         return 0.0, 0.0
9
10    area_interseccion = (x_der - x_izq) * (y_abajo - y_arriba)
11    area_box1 = (box1[2] - box1[0]) * (box1[3] - box1[1])
12    area_box2 = (box2[2] - box2[0]) * (box2[3] - box2[1])
13
14    if area_box1 <= 0 or area_box2 <= 0:
15        return 0.0, 0.0
16
17    area_union = area_box1 + area_box2 - area_interseccion
18    if area_union <= 0:
19        return 0.0, 0.0
20
21    iou = area_interseccion / float(area_union)
22
23    area_minima = min(area_box1, area_box2)
24    contencion = area_interseccion / float(area_minima)
25
26    return iou, contencion
```

Listing 13: Cálculo de IoU y contención entre dos cajas.

El IoU se calcula como el área de intersección entre dos cajas dividida entre el área de su unión. En esta implementación se utiliza un umbral de IoU igual a 0.2. Además, se calcula una medida de contención dividiendo el área de intersección entre el área de la caja más pequeña. Si esta proporción supera 0.6, se considera que una caja está contenida dentro de otra y se elimina.

```
1 while len(detecciones) > 0:
2     mejor_det = detecciones.pop(0)
3     detecciones_finales.append(mejor_det)
4
5     detecciones_restantes = []
```

```
6     for det in detecciones:
7         iou, contencion = calcular_iou_y_contencion(mejor_det['box'],
8             det['box'])
9
10        if iou < umbral_iou and contencion < umbral_contencion:
11            detecciones_restantes.append(det)
12
13    detecciones = detecciones_restantes
```

Listing 14: Supresión de detecciones repetidas mediante IoU y contención.

Este segundo criterio es especialmente útil en este problema, ya que algunas detecciones pueden corresponder a símbolos, flechas o zonas internas del panel. Gracias a la combinación de IoU, contención y puntuación ponderada por área, el sistema conserva la caja más representativa y reduce el número de detecciones duplicadas.

2.6. Formato de salida y scripts de apoyo

El programa genera un fichero `resultado.txt` compatible con el formato indicado en el enunciado de la práctica. Cada línea contiene una detección con la siguiente estructura:

`nombre_imagen;x1;y1;x2;y2;clase;score`

Donde `x1` e `y1` son las coordenadas de la esquina superior izquierda de la caja, `x2` e `y2` corresponden a la esquina inferior derecha, la clase se fija siempre a 1 y el `score` representa la confianza asignada por el detector.

```
1 linea_txt = f"{nombre_img};{x1};{y1};{x2};{y2};1;{score:.3f}\n"
2 f_txt.write(linea_txt)
```

Listing 15: Escritura de detecciones en resultado.txt.

Además, el script `main.py` crea el directorio `resultado_imgs`, donde se guardan las imágenes de test con las detecciones dibujadas. Las cajas se representan en color rojo y la puntuación asociada se muestra en amarillo, siguiendo el formato solicitado.

```
1 cv2.rectangle(img, (x1, y1), (x2, y2), (0, 0, 255), 2)
2 cv2.putText(img, f"{score:.2f}", (x1, y1 - 10),
3             cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)
```

Listing 16: Dibujo de cajas y puntuaciones sobre la imagen.

El script de evaluación permite indicar tanto el fichero de resultados generado como el fichero de referencia del profesor. También permite mostrar las detecciones sobre las imágenes de test si se activa el argumento correspondiente.

```
python evaluar_resultados.py --test_path test_detection \  
    --detections_file resultado.txt \  
    --instructor_detections_file resultado_jmbuena_road_panels.txt
```

Listing 17: Comando de evaluación de resultados.

3. Evaluación de los resultados obtenidos

Para evaluar el rendimiento del detector se ha utilizado el script `evaluar_resultados.py` proporcionado en la práctica. Este script compara las detecciones generadas en `resultado.txt` con las anotaciones del conjunto de test y calcula las curvas *precision-recall*.

La evaluación se ha realizado con dos umbrales de IoU. El primer umbral, $\text{IoU} > 0,5$, permite medir si las detecciones se encuentran aproximadamente sobre los paneles anotados. El segundo umbral, $\text{IoU} > 0,7$, es más exigente y evalúa si las cajas generadas son suficientemente precisas en posición y tamaño.

La métrica utilizada para resumir el comportamiento del detector es AP, *Average Precision*. Un valor de AP más alto indica que el detector mantiene una mejor relación entre precisión y *recall* a lo largo de distintos umbrales de *score*.

3.1. Resumen cuantitativo

Tras procesar la totalidad de las imágenes de test, el detector alcanza un rendimiento superior a los valores de referencia indicados. La comparación de precisión media se resume en la Tabla 1.

Tabla 1: Comparación de AP frente a la referencia.

Métrica	Referencia	Solución desarrollada
AP con $\text{IoU} > 0,5$	68.7	79.0
AP con $\text{IoU} > 0,7$	60.9	77.6

Para $\text{IoU} > 0,5$, la solución alcanza un AP de 79.0 frente al 68.7 de la referencia. Esto indica que el detector consigue localizar correctamente una proporción elevada de paneles manteniendo una precisión competitiva.

Para $\text{IoU} > 0,7$, la solución obtiene un AP de 77.6 frente al 60.9 de la referencia. Esta diferencia refleja que el ajuste de las cajas es especialmente bueno, probablemente debido a la ampliación controlada de las regiones MSER, la validación final por densidad de bordes y el filtrado mediante NMS.

3.2. Curvas precision-recall

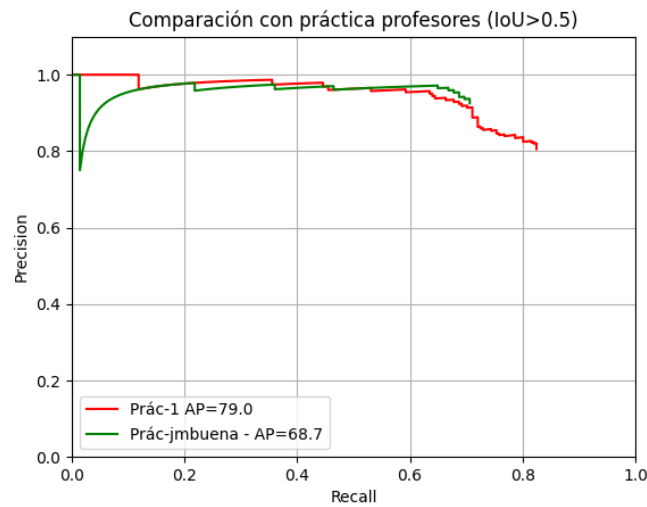


Figura 2: Curva precision-recall obtenida con umbral IoU > 0,5.

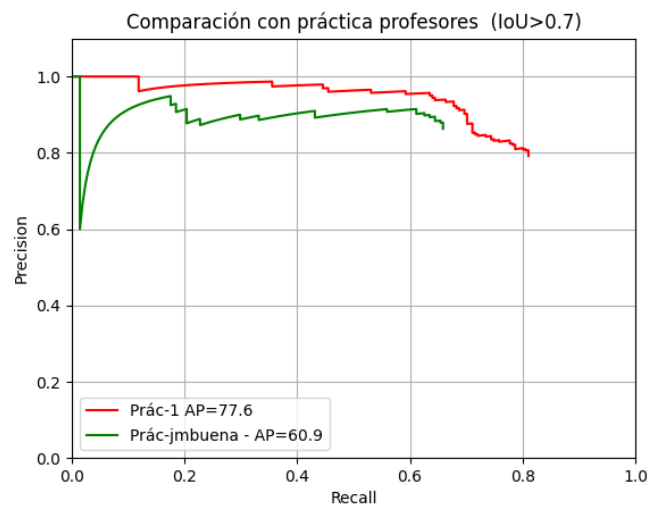


Figura 3: Curva precision-recall obtenida con umbral IoU > 0,7.

Las curvas *precision-recall* permiten analizar el comportamiento del detector para distintos valores de score. En general, una curva situada en la zona superior del gráfico indica que el detector mantiene una precisión elevada incluso cuando aumenta el *recall*. En este caso, los valores de AP obtenidos muestran que el sistema no solo detecta un número considerable de paneles, sino que además mantiene un número controlado de falsos positivos.

3.3. Imágenes de evaluación



Figura 4: Ejemplo de detección en una escena con buena visibilidad.



Figura 5: Ejemplo de detección en una escena con baja saturación o niebla.

En las imágenes de salida se observa que el detector localiza correctamente la mayoría de paneles presentes en la zona superior de la escena. Las cajas rojas delimitan las regiones aceptadas como paneles y el texto amarillo indica el *score* asignado a cada detección.

En escenas con buena iluminación, el detector principal basado en MSER y HSV obtiene resultados precisos, ya que el color azul del panel se encuentra claramente separado del fondo. En escenas más complejas, como imágenes con niebla, bajo contraste o paneles parcialmente borrosos, los detectores complementarios permiten recuperar algunas regiones que no serían detectadas únicamente con el umbral de azul saturado.

3.4. Discusión de resultados

Los resultados obtenidos muestran que la combinación de MSER y validación HSV es adecuada para este problema. MSER permite proponer regiones candidatas basadas en contraste, mientras que la máscara HSV actúa como filtro específico para comprobar si la región contiene una proporción suficiente de azul.

El principal punto fuerte del sistema es que no depende de aprendizaje automático ni de entrenamiento supervisado, sino de propiedades visuales directamente relacionadas con los paneles: color, forma, contraste, posición y textura interna. Esto hace que el detector sea sencillo e interpretable.

La mejora obtenida en $\text{IoU} > 0,7$ indica que el ajuste de las cajas es preciso. En este aspecto influye la ampliación del 8 % aplicada a las cajas MSER, ya que permite incluir el borde blanco externo del panel. También contribuye el algoritmo NMS, que elimina detecciones internas o repetidas y conserva normalmente la caja con mejor puntuación ponderada.

A pesar de los buenos resultados, el sistema presenta algunas limitaciones. En imágenes con iluminación muy variable, reflejos, niebla intensa o paneles muy pequeños, el color azul puede no ajustarse al rango HSV definido. En esos casos, el detector puede perder algunos paneles o aceptar regiones falsas con tonos similares. También pueden aparecer falsos positivos en zonas del cielo o en objetos de color azulado, aunque los filtros geométricos, la validación por densidad de bordes y el NMS reducen este problema.

En conjunto, el detector ofrece un buen equilibrio entre precisión y *recall*. El resultado es especialmente positivo teniendo en cuenta que solo se han utilizado técnicas clásicas de procesamiento de imagen, sin clasificadores ni modelos de aprendizaje automático.

4. Conclusión

En esta práctica se ha desarrollado un sistema completo de detección de paneles informativos de autopista mediante técnicas clásicas de visión artificial. La solución combina un detector principal basado en MSER con una validación por color en el espacio HSV, filtros geométricos, estrategias complementarias basadas en bordes y contornos, una validación final por densidad de bordes y un postprocesado mediante NMS para eliminar detecciones repetidas.

El sistema cumple los requisitos establecidos en el enunciado: procesa las imágenes de test, genera el fichero `resultado.txt` con el formato solicitado, guarda las imágenes con las detecciones dibujadas y permite evaluar el rendimiento mediante las curvas *precision-recall* proporcionadas por el script oficial.

Los resultados obtenidos son satisfactorios, con valores de AP superiores a la referencia tanto para $\text{IoU} > 0,5$ como para $\text{IoU} > 0,7$. Esto indica que el detector no solo localiza correctamente gran parte de los paneles, sino que además genera cajas suficientemente

precisas.

Como posibles mejoras futuras, se podrían estudiar nuevas técnicas dentro del marco de procesamiento clásico de imagen, como la detección de líneas mediante transformada de Hough, un ajuste más adaptativo de los umbrales HSV o una estimación automática del color dominante de los paneles a partir del conjunto de entrenamiento.

5. Bibliografía

Referencias

- [1] Universidad Rey Juan Carlos. *Práctica obligatoria 1: Detección de objetos*. Asignatura Visión Artificial, 2026.
- [2] OpenCV. *OpenCV Documentation*. Documentación oficial de OpenCV. Disponible en: <https://docs.opencv.org/>
- [3] OpenCV. *cv::MSER Class Reference*. Documentación oficial de OpenCV. Disponible en: https://docs.opencv.org/3.4/d3/d28/classcv_1_1MSER.html
- [4] OpenCV. *MSER Python sample*. Repositorio oficial de OpenCV en GitHub. Disponible en: <https://github.com/opencv/opencv/blob/master/samples/python/mser.py>
- [5] OpenCV. *Finding contours in your image*. Tutorial oficial de OpenCV. Disponible en: https://docs.opencv.org/3.4/df/d0d/tutorial_find_contours.html
- [6] OpenCV. *Morphological Transformations*. Tutorial oficial de OpenCV. Disponible en: https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html
- [7] OpenCV. *Canny Edge Detection*. Tutorial oficial de OpenCV. Disponible en: https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html
- [8] OpenCV. *Changing Colorspaces*. Tutorial oficial de OpenCV. Disponible en: https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html
- [9] OpenCV. *Operations on arrays: inRange*. Documentación oficial de OpenCV. Disponible en: https://docs.opencv.org/4.x/d2/de8/group__core__array.html
- [10] OpenCV. *OpenCV source code*. Repositorio oficial en GitHub. Disponible en: <https://github.com/opencv/opencv>
- [11] Belval. *opencv-mser: A working example of OpenCV 3 MSER detector*. Repositorio en GitHub. Disponible en: <https://github.com/Belval/opencv-mser>